

Liquid Glass, simply.

Glass for Unity uGUI on Universal Render Pipeline. Choose an outline, shape the light, and combine surfaces through an intentional fusion group.

Install once. Start with one component.

1. Use Unity 6.3 LTS with URP 17.3, RenderGraph enabled and Unity UI 2.0. Import all product files. For demos, install Input System, enable New or Both in Active Input Handling, and choose **Window > TextMeshPro > Import TMP Essential Resources**.
2. Open **Tools > Translucent UI FX > Setup & Diagnostics**. Install the renderer feature on your active URP renderer. Check Graphics and Quality pipeline overrides. Importing does not change your project pipeline settings.
3. Choose **GameObject > UI > Liquid Glass Panel**, or add **UI > Liquid Glass** to an empty UI RectTransform. Resize it and adjust Tint, Blur or Refraction. It is an Image itself; no material, sprite or camera reference is needed for ordinary panels.

Everyday controls first

Area	Purpose
Appearance	Tint, blur, refraction, highlights and optional pointer lighting.
Shape / Fusion	Eight shapes or a sprite outline; join members, blend distance and cutouts.
Optics / Lighting	Optional lens, lip, color dispersion, light bands and shadows.
Background	Automatic background, quality and refresh. Special camera/layer overrides remain under Advanced.
Image & Interaction	Native Image settings, masking and shape-aware pointer hit areas.

Try the playground

Open **Demo/Scenes/Demo - Liquid Glass** and press Play. Try the sliders and shapes; drag the fusion members together or apart. Enable the cutout and pointer lighting. The second scene, **Demo - Fused Liquid Glass**, shows another composition. Radius is disabled for sprites and shapes without adjustable corners.

Shape. Join. Cut.

Eight starting points

Continuous, Rounded, Capsule, Circle, Rectangle, Diamond, Triangle and Hexagon. Circle stays circular in a wide rectangle; Capsule follows the longer axis. Rounded and Continuous support a simple radius and independent corners.

Custom corner profiles add absolute or relative sizes, opposing-corner normalization and independent curvature. Continuous profiles range from round to squircle. Geometric profiles range from round through chamfer to concave. Pointer hit areas follow the silhouette.

Use your own sprite

Assign **Shape sprite**. Alpha defines the outline and refractive boundary; sprite RGB does not color the glass. Texture Read/Write is not required. Atlas regions, packed orientation and pivots are handled through sprite mesh and UV extraction. Clear the sprite to return to procedural geometry.

Alpha threshold selects the optical contour. Standalone Images retain their original soft alpha. Shape detail offers 64, 128, 256, 512 and 1024; start at the default 256. Matching sprites share their retained distance fields.

Compose one surface

1. Select sibling Liquid Glass objects and choose **Fuse shapes**, or create a group from one object.
2. Move, rotate, resize or duplicate members. Only members of that group merge.
3. Adjust **Blend distance**: zero gives a sharp union; larger values form soft bridges.
4. Add a **Cutout** to remove glass and its pointer hit area.

Sprite and procedural members can mix. Capacity grows dynamically, but cost grows with covered pixels and member count. Split large or widely separated collections. The Composition map shows geometry, including cutouts; click a numbered member to select it.

Keep your existing UI

Conversion records one Undo operation and preserves transforms, child content, appearance and Button/Toggle graphic references. Detach as standalone glass retains edited shapes and appearance. Filled, sliced and tiled Images remain standalone; use Simple for sprite fusion.

Members inherit the group appearance or override tint, graphic color, blur, fade, optics and lip bands; values blend across joins. The group supplies common light and shadow. Keep group Raycast Target off and put controls on members. Nested groups own their own members. Under RectMask2D, keep the group layout rectangle covering its visible members.

Control the finish.

A clear center and a refractive edge

Refraction controls bending strength; zero turns bending off. Optical lip sets boundary width in Canvas units or a percentage of either side. Smoothing softens optical normals without changing the silhouette. Interior stabilization keeps broad joined surfaces calm.

Lens depth and refractive index shape the bend. Dispersion separates color channels. Magnification zooms the background around the shape center; Transmission scales its brightness. Opacity / fade fades the complete surface.

Light follows the shape

Directional uses a direction pad. Opposing adds a second light. Point uses viewport coordinates: (0, 0) is bottom-left and (1, 1) is top-right. Follow pointer is optional and needs Raycast Target enabled on the receiving graphic.

Inner and outer lip bands accept fractions of the optical lip or Canvas units. Light and complementary shade have independent color alpha. Drop shadows use screen-pixel size and offset; zero color alpha disables them. Wide shadows increase covered pixels and cost.

Preserve quality; avoid unnecessary work

Setting	Tradeoff
Background quality	High retains a full-resolution source. Medium and Low reduce bandwidth. Matching panels share the strongest request.
Shape detail	Higher sprite resolution preserves fine details but costs more generation time and memory. Procedural shapes need no sprite field.
Renderer tuning	Optional mip levels, blur passes, kernel spread, dithering and color format. Unsupported formats fall back to the camera format.
Camera override	Optional per-camera quality and blur settings for unusual rendering arrangements.

Refresh and sprite generation

Real-time follows animated backgrounds. **Camera changes** detects camera/projection changes and explicit requests, not arbitrary animation. **Every few frames** reduces refresh frequency; **Manual** retains the image until RequestUpdate(). Use Real-time for ordinary moving games.

A new sprite field uses GPU alpha extraction, synchronous readback, a CPU distance transform and upload. Avoid many new high-resolution sprites in one latency-sensitive frame. Moving shapes reuse their fields. Texture updates and Editor imports invalidate fields; call GlassSpriteField.InvalidateAll() after runtime sprite geometry changes.

The world behind it.

What “background” means

Glass needs a temporary copy of the rendered game image to blur and bend light. This stays in GPU memory; nothing is recorded or saved. Panels using the same camera and layer share the source and blurred image.

Automatic for ordinary panels

Camera-space and world-space UI use their Canvas camera. Overlay UI prefers a suitable Main Camera, then an active game camera on the same display, favoring a full scene view over minimaps or UI-only views. Automatic selection excludes render-to-texture and URP Overlay cameras and follows camera switches.

Without an available background, the surface temporarily shows its translucent tint. Optics resume when the background becomes available. The inspector checks the selected camera renderer and offers setup if the feature is missing or disabled.

Advanced routing only when needed

Background > Advanced background override provides an optional camera and layer. Add **Liquid Glass Capture Source** to a Canvas or ancestor to share an override across a subtree. A **Liquid Glass Camera Override** can tune quality for one camera. Multiple cameras retain independent background images.

Rendering order matters

Glass sees only content drawn before its background pass. Use **Before transparents** for camera/world-space glass on the source camera. Use **After transparents** for Overlay glass above scene transparency. Overlay sibling graphics are drawn too late to be included in that image.

Glass over glass

For two levels on one camera, add two renderer features with distinct layer numbers. Layer 0 captures before transparents; draw the back camera-space Canvas; layer 1 captures after transparents and includes that Canvas. Front Overlay glass uses layer 1. More camera-space levels require explicit rendering or camera ordering.

Game View is the optical preview

Scene View provides a shape and tint authoring preview. Use Game View or a player build to judge refraction, blur and background ordering. The background image covers the camera view; a separate capture-padding control is unnecessary.

Ready for your project.

Scripting

```
using TranslucentUIFX;
glass.ApplyLiquidGlass();
glass.BlurStrength = 0.12f;
glass.sprite = myShapeSprite;
glass.SetAllDirty();
TranslucentRendererFeature.RequestUpdate();
```

Clear both sprite and overrideSprite when switching to procedural geometry in code. Runtime lives in TranslucentUIFX.Runtime; custom inspectors and authoring tools live in the Editor-only assembly. Old named preset IDs resolve to Liquid Glass; Custom leaves existing values intact.

Troubleshooting

Only a flat tint: expand Background, verify the selected renderer has the enabled feature, check Graphics and Quality pipeline overrides, and enable RenderGraph. Ordinary game panels need no camera assignment.

Frozen background: choose Real-time refresh.

Missing UI behind glass: use camera-space/layered rendering; Overlay siblings are drawn too late.

No sprite bending: raise Refraction, check sprite alpha, and inspect Game View.

Old inspector: fix compilation errors, import all updated files and remove duplicate package copies. Obsolete Tests left by development exports can be removed in the receiving project.

No demo input: install Input System, enable New or Both, and press Play.

Upgrade and reset

Back up your project and import all files into the original asset folder so GUIDs remain intact. The full package updates its included demo scenes. Your own game scenes are not included. Reset appearance preserves shapes, sprites and background settings; the playground Reset also restores fusion positions.

Requirements and scope

Release target: **Unity 6.3 LTS / URP 17.3 / uGUI 2.0**. Demos use Input System 1.20.0. RenderGraph, shader-model-4.5-class hardware, structured buffers, texture arrays and RFloat textures are required. Resources assets retain the runtime shaders for player builds. Built-in, HDRP, UI Toolkit and WebGL are unsupported. Mobile, XR and other graphics APIs require separate validation. Profile your target device; Editor timings are not a universal FPS guarantee.

Offline help and support

Read **README.md** for integration and **IMPORT AND TEST.md** for upgrades. Inter uses SIL Open Font License 1.1; see **Third-Party Notices.txt**. For help, open **Setup & help** or visit discord.gg/bXABNPthTb. Include Unity/URP versions, graphics API, Canvas mode and a minimal reproduction.